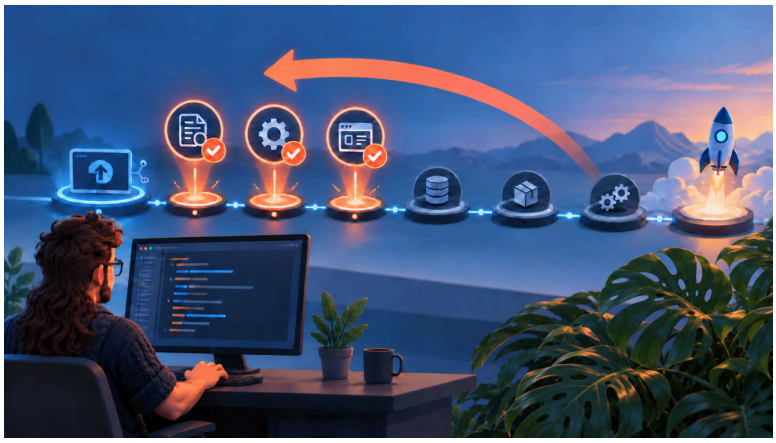
Edit article
View post



Generated by Codex

Shifting Left? #ABAP

Lars Hvam Petersen
Senior SAP/ABAP Consultant at Heliconia Labs

October 20, 2026

So, what does "Shifting Left" actually mean? Here are a few scribblings on how I see the concept. Feel free to comment below with your views and opinions.

Wikipedia defines it as: *"Shift-left testing is an approach to software testing and system testing in which testing is performed earlier in the lifecycle."*

I see it as **providing feedback to the team as early as possible**. That feedback should, in the end, give us better software earlier, and it can involve lots of different steps.

Step 1 - Visibility

The full team needs visibility to collaborate. What is the latest feedback? What changes are ongoing? Put the documentation and code changes in one common place that the full team can access, and make sure it's versioned so everyone can see exactly what happened.

This allows developers to publish their code and get early, automated feedback. It also allows other team members to check the progress of changes and help influence them early, instead of possibly weeks later.

Step 2 - Static Analysis

Some of the easiest and fastest feedback comes from static analysis. Set it up so every code change is checked. Then a developer can easily bisect the history to identify when something broke.

This also ties in with the Continuous Integration concept. As an example, abaplint works great for this step.

Step 3 - Unit Tests

Next, write unit tests! Have all of them run on each code change. It might take a few minutes, but that's fine—much better than forgetting to run them and leaving failing tests behind.

With a shifted-left setup, there is nothing for the developer to remember. Ideally, all quality checks run early, making it possible to identify exactly which change caused something to fail. The developer can then figure it out and fix it while the change is still fresh.

Also see my blog from 2021 on **how to run unit tests** on GitHub Actions.

Step 4 - UI and e2e tests

Test the user interfaces. Spin up external systems in Docker and check that integrations work end to end.

Have the test cases follow the code changes. If a change breaks a test, it needs to be fixed, and during review it's possible to see whether any tests were added.

Anti-patterns

It's 2026. Organizations can define how they want to work, and LLMs make many things possible, but they also challenge how we do things. I see a few typical anti-patterns:

Feedback on transport release is too late. The full team needs early feedback. What if the change breaks tests or doesn't follow naming conventions? It can take a long time to refactor, even though the work is "done". Shift left, get early feedback, and release with confidence!

Manually triggering static analysis and unit tests. Developers should not need to remember to trigger these checks. Just publish the changes to the platform and get a backup of the changes, along with all quality checks triggered automatically.

Reviewing moving state. Always review versioned code. Code in an ABAP system can be versioned by releasing a transport, but it might be changing during review. Put the code under proper versioning so both developer and reviewer look at *exactly* the same state.

Tests triggered weekly in the Quality system will lead to a never-ending blame game. Who broke something? When did it break? Which transport was it? Was it a transport, or was it master data? Shift left and identify the defect early. More testing is always better, so testing in Quality probably needs to happen in any case. But consider it a feedback loop: when something breaks in Quality, make sure you can get that feedback earlier next time.

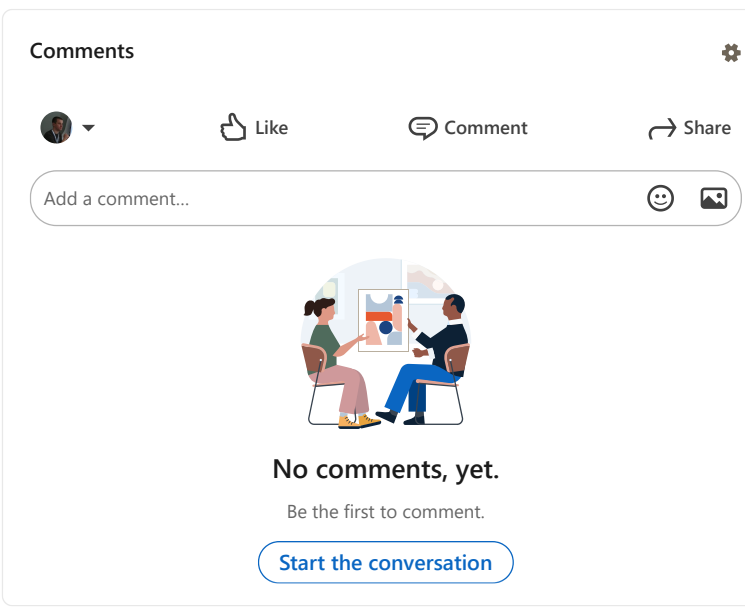
But how?!

Put the ABAP code into **GitHub** or something similar. I'm sure organizations that run SAP systems already have an approved common platform for versioning code that much of the organization uses? Then start setting up Continuous Integration, improving one step at a time.

Consider the benefits of inner sourcing, and try making ABAP/SAP development normal by following the same established development practices as the rest of the organization. And lots of **more benefits await**—it's just a matter of getting started!

Oh, and these setups also works great for LLMs, providing them feedback, and the teams get visibility into what is the state and which are the changes.







Lars Hvam Petersen

Senior SAP/ABAP Consultant at Heliconia Labs